

CS338 Introduction to Theory of Computation 详细复习 Note

2026 年 6 月 14 日

目录

0. 考试答题规范：不要只写结论	4
0.1 Construction proof 必须写完整构造	4
1. DFA 与正则语言	6
1.1 DFA 定义与接受	6
1.2 设计 DFA 的常用状态含义	6
1.3 正则运算和闭包	6
1.4 Assignment 1 对齐	6
2. NFA、正则表达式、GNFA	7
2.1 NFA 定义	7
2.2 NFA 与 DFA 等价	7
2.3 NFA 转 DFA: subset construction 完整写法	8
2.4 NFA 转 DFA 示例 1: 无 ϵ 的 NFA	8
2.5 NFA 转 DFA 示例 2: 带 ϵ -closure	9
2.6 正则表达式	9
2.7 Thompson 构造与状态消除	10
2.8 Assignment 2 对齐	10
2.9 Assignment 3 对齐	10
3. 非正则语言与 Pumping Lemma	11
3.1 正则 Pumping Lemma	11
3.2 Assignment 4 标准证明	11
4. CFG、CFL、PDA	12
4.1 CFG 定义	12
4.2 常用 CFG	12
4.3 CNF	13
4.4 PDA	13
4.5 CFG 转 PDA: 完整构造	13
4.6 CFG 转 PDA 示例: 0^n1^n	14
4.7 PDA 转 CFG: 完整构造	15

4.8 PDA 转 CFG 示例: 从 PDA 得到 $S \rightarrow 0S1 \mid \epsilon$	16
4.9 CFL 闭包与非闭包	17
4.10 CFL Pumping Lemma	17
4.11 CNF 推导长度	17
5. Turing Machines 与 Church-Turing	18
5.1 TM 定义	18
5.2 Decider vs Recognizer	18
5.3 TM 变体	18
5.4 TM 设计算法题	19
6. Decidability 与 Undecidability	19
6.1 可判定语言	19
6.2 Assignment 8 对齐	19
6.3 Diagonalization 与 A_{TM}	20
6.4 Reducibility	20
6.5 Rice Theorem	21
6.6 Assignment 9-10 对齐	21
7. Time Complexity 与 P	22
7.1 时间复杂度	22
7.2 P 中的典型问题	22
7.3 P 闭包与 Assignment 10	22
8. NP、coNP、Polynomial Reducibility	23
8.1 NP 定义	23
8.2 常见 NP 证明	23
8.3 NP 闭包	23
8.4 P、NP、coNP 判断题	23
8.5 2-COLOR 与 2-SAT	24
9. NP-Completeness	24
9.1 定义与证明套路	24
9.2 3SAT $\rightarrow$ CLIQUE	24
9.3 2SAT $\rightarrow$ CLIQUE	25
9.4 Double-SAT NP-complete	25
9.5 3SAT $\rightarrow$ HAMPATH	25
9.6 Directed HAMPATH $\rightarrow$ Undirected HAMPATH	25
9.7 3SAT $\rightarrow$ VERTEX-COVER	26
9.8 3COLOR NP-complete	26
10. Cook-Levin 与 3SAT	27

10.1 Cook-Levin Theorem	27
10.2 Tableau	27
10.3 为什么用 2×3 window	27
10.4 Assignment 13 Q1 tableau	27
10.5 SAT $\rightarrow$ 3SAT	28
11. Space, PSPACE, NP-hardness, 近似	28
11.1 Space complexity	28
11.2 NP-hardness	29
11.3 近似与随机算法	29
12. 快速总表	29
12.1 模型能力	29
12.2 常见 “可判定/不可判定/可识别”	30
12.3 常见 NP-complete 链	30
12.4 作业对应复习索引	30
13. 最后复习提醒	31

资料覆盖:ppts/ 中全部课件,Week1 至 Week13 的作业题面与 LaTeX/PDF 答案,answers/ 中 Assignment 1-13 官方答案。

使用方式: 先看第 0 节答题规范, 再按 DFA/NFA/RE -> CFL/PDA -> TM/Decidability -> Complexity 的顺序复习。

0. 考试答题规范: 不要只写结论

0.1 Construction proof 必须写完整构造

如果题目说 “use the construction proof”, 答案必须给出新机器/新文法的完整形式, 不能只画直觉图或说 “显然封闭”。

DFA 乘积构造模板 (交、并、差、对称差)

给定

$$M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1), \quad M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2).$$

构造

$$M = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F)$$

其中

$$\delta((r, s), a) = (\delta_1(r, a), \delta_2(s, a)).$$

按目标语言设置接受态:

- 交集: $F = F_1 \times F_2$
- 并集: $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$
- 差集: $F = F_1 \times (Q_2 - F_2)$
- 对称差: $F = (F_1 \times (Q_2 - F_2)) \cup ((Q_1 - F_1) \times F_2)$

最后写一句正确性: 读入同一前缀后, 第一分量正是 M_1 的状态, 第二分量正是 M_2 的状态, 因此接受条件正好等价于目标布尔组合。

NFA 并/连接/星号构造模板

- 并: 新起始态 q_0 , 从 q_0 用 ϵ -transition 到两个旧起始态; 接受态为两个 NFA 的接受态并集。
- 连接: 从第一个 NFA 的每个接受态用 ϵ -transition 到第二个 NFA 的起始态; 接受态为第二个 NFA 的接受态。
- Star: 新起始态也是接受态; 从新起始态 ϵ 到旧起始态; 从每个旧接受态 ϵ 回旧起始态。
- 单接受态: 新增唯一接受态 q_a , 从所有旧接受态 ϵ 到 q_a , 原接受态取消接受。

子集构造模板

给 NFA $N = (Q, \Sigma, \delta, q_0, F)$, 构造 DFA $D = (2^Q, \Sigma, \delta', E(q_0), F')$ 。

若有 ϵ -transition, 则

$$\delta'(S, a) = E \left(\bigcup_{q \in S} \delta(q, a) \right).$$

接受态:

$$F' = \{S \subseteq Q \mid S \cap F \neq \emptyset\}.$$

作业里需要列出可达子集和转移表, 不要只写“用 subset construction”。

CFG $\rightarrow$ PDA 模板

构造 PDA 用栈模拟最左推导:

1. 初始先压入底符号 \$, 再压入起始变量 S 。
2. 对每条产生式 $A \rightarrow u$, 加 $\epsilon, A \rightarrow u^R$ 的替换路径 (多符号压栈要拆成多个中间状态)。
3. 对每个终结符 a , 加读入并匹配的转移 $a, a \rightarrow \epsilon$ 。
4. 栈底 \$ 被弹出且输入读完时接受。

PDA $\rightarrow$ CFG 模板

先保证 PDA 只有一个接受态且接受前清空栈。变量 A_{pq} 表示从状态 p 到 q 并清空所压栈内容的所有字符串:

- 基础规则: $A_{pp} \rightarrow \epsilon$
- 连接规则: $A_{pq} \rightarrow A_{pr}A_{rq}$
- 匹配规则: 若 $p \xrightarrow{a, \epsilon \rightarrow t} r \text{ push } t$, 且 $s \xrightarrow{b, t \rightarrow \epsilon} q \text{ pop } t$, 则

$$A_{pq} \rightarrow aA_{rs}b.$$

归约证明模板

要证明目标 B 难, 必须从已知难问题 A 归约到 B : $A \leq_m B$ 或 $A \leq_p B$ 。写法固定:

1. 说明已知 A 的性质, 例如 A_{TM} undecidable, 或 3SAT NP-complete。
2. 给出输入 x 到实例 $f(x)$ 的构造, 且构造可计算/多项式时间。
3. 证明双向:

$$x \in A \iff f(x) \in B.$$

4. 结论: 若 B 可解, 则 A 可解, 矛盾; 或 B NP-hard。若还要 NP-complete, 必须再证明 $B \in NP$ 。

Pumping lemma 答题模板

证明非正则:

1. 假设 L 正则, 令 p 为 pumping length。
2. 选 $w \in L$ 且 $|w| \geq p$ 。
3. 任取分解 $w = xyz$, 满足 $|xy| \leq p, |y| > 0$ 。
4. 由 $|xy| \leq p$ 定位 y 在某一块内。
5. 选 $i = 0$ 或 $i = 2$, 证明 $xy^iz \notin L$, 矛盾。

证明非 CFL:

1. 假设 L 是 CFL, 令 p 为 CFL pumping length。
2. 选 $s \in L$, 分解 $s = uvxyz$, 满足 $|vxy| \leq p, |vy| \geq 1$ 。
3. 由于 $|vxy| \leq p$, 它最多跨越有限相邻块。

4. 分情况或统一说明 pumping 改变局部块，破坏全局相等/比例关系。
5. 选 $i = 0$ 或 $i = 2$ ，得到 $uv^i xy^i z \notin L$ 。

1. DFA 与正则语言

1.1 DFA 定义与接受

DFA 是五元组

$$M = (Q, \Sigma, \delta, q_0, F)$$

其中 Q 是有限状态集， Σ 是字母表， $\delta: Q \times \Sigma \rightarrow Q$ ， q_0 是初始态， $F \subseteq Q$ 是接受态。

DFA 接受 $w = w_1 \cdots w_n$ ，当且仅当存在状态序列

$$r_0 = q_0, \quad r_{i+1} = \delta(r_i, w_{i+1}), \quad r_n \in F.$$

语言 $L(M) = \{w \mid M \text{ accepts } w\}$ 。若某语言被某 DFA 识别，则它是 regular language。

1.2 设计 DFA 的常用状态含义

复习时不要背图，背“状态记录什么信息”：

- 以某串结尾：状态记录最长后缀匹配了目标串多少位。例如 contains 0101，状态记录已匹配 0, 01, 010, 0101。
- 至少/至多出现次数：状态记录计数并在阈值处吸收，例如至少四个 1 用 0, 1, 2, 3, ≥ 4 五类。
- 奇偶性：两个状态来回切换。
- “以 0 开头且以 1 结尾”：先记住开头是否合法，再记录最后一个字符。
- “不含某 substring”：先构造“含该 substring”的 DFA，再补集；或直接设置陷阱态。
- 交集语言：先构造两个简单 DFA，再做乘积构造。

1.3 正则运算和闭包

正则语言对并、交、补、差、连接、Kleene star 封闭。

- 并/交/差/补：DFA 层面最直接，使用乘积或翻转接受态。
- 连接/星号：NFA 层面最自然，使用 ϵ -transition。
- 反转 A^R ：正则表达式结构归纳最直接：

$$(R_1 \cup R_2)^R = R_1^R \cup R_2^R, \quad (R_1 R_2)^R = R_2^R R_1^R, \quad (R^*)^R = (R^R)^*.$$

1.4 Assignment 1 对齐

官方答案要求会把图转为形式化描述。写 DFA 形式化描述时必须列：

- Q
- Σ

- δ 表格
- q_0
- F

作业中的典型 DFA 语言：

- begins with 0 and ends with 1
- contains at least four 1s
- contains substring 0101
- length at least 4 and fourth symbol is 1
- starts with 1 and has odd length, or starts with 0 and has even length
- contains at least two 0s and at most one 1
- empty language
- all strings except ϵ

补集构造题：

- 不含 **ba**：先构造含 **ba** 的 DFA，再翻转接受态。
- 既不含 **ab** 也不含 **ba**：等价于 $a^* \cup b^*$ ，也可先构造含 **ab** 或 **ba** 再补。
- 不在 b^*a^* 中：等价于含 substring **ab**，注意方向不要写反。旧答案中题面与文字有编码混乱，复习时按答案源码的构造理解。

交集构造题：

- 至少两个 **a** 且至少三个 **b**：状态 (i, j) ，其中 $i \in \{0, 1, \geq 2\}$ ， $j \in \{0, 1, 2, \geq 3\}$ ，接受 $(\geq 2, \geq 3)$ 。
- 偶数个 **a** 且一个或两个 **b**：状态记录 **a** parity 和 **b** count $0, 1, 2, \geq 3$ ，接受 $(\text{even}, 1)$ 与 $(\text{even}, 2)$ 。
- 以 **a** 开头且至多两个 **b**：可以先判断首字符，再计数 **b**。

2. NFA、正则表达式、GNFA

2.1 NFA 定义

NFA 仍是五元组，但转移函数为

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q).$$

NFA 接受一个串，只要存在一条路径，路径标签（去掉 ϵ ）等于该串且终点在接受态。

2.2 NFA 与 DFA 等价

核心定理：若 NFA 识别 A ，则 A 正则。证明用 subset construction。

有 ϵ -transition 时必须使用 ϵ -closure。

2.3 NFA 转 DFA: subset construction 完整写法

考试或作业要求“convert the following NFA to a DFA”时，不能只写“由 subset construction 得”。答案要包含四件事：每个 DFA 状态代表的 NFA 状态集合、起始集合、转移表、接受集合。

给定

$$N = (Q, \Sigma, \delta, q_0, F).$$

若 N 有 ϵ -transition，先定义

$$E(R) = \{q \in Q \mid q \text{ can be reached from some } r \in R \text{ using only } \epsilon\text{-moves}\}.$$

若没有 ϵ -transition，可把 $E(R) = R$ 。

构造 DFA

$$D = (Q_D, \Sigma, \delta_D, q_D, F_D)$$

如下：

1. 起始态：

$$q_D = E(\{q_0\}).$$

2. 对每个已经发现的子集状态 $S \subseteq Q$ 和每个输入符号 $a \in \Sigma$ ，计算

$$\text{Move}(S, a) = \bigcup_{q \in S} \delta(q, a),$$

再令

$$\delta_D(S, a) = E(\text{Move}(S, a)).$$

3. 若新集合还没出现，把它加入待处理队列。只列可达子集即可；若题目要求 complete DFA，则 $\emptyset$ 也要作为死状态列出。
4. 接受态是所有包含旧接受态的子集：

$$F_D = \{S \in Q_D \mid S \cap F \neq \emptyset\}.$$

正确性句子可以固定写：对任意输入串 w ，DFA 读完 w 后所在的子集，恰好等于 NFA 从 q_0 读完 w 后可能处在的所有状态的 ϵ -closure。因此该子集与 F 相交，当且仅当 NFA 有某条接受路径。

2.4 NFA 转 DFA 示例 1：无 ϵ 的 NFA

构造一个识别“以 01 结尾”的 NFA：

$$Q = \{q_0, q_1, q_2\}, \quad \Sigma = \{0, 1\}, \quad q_0 \text{ start}, \quad F = \{q_2\}.$$

转移为：

$$\delta(q_0, 0) = \{q_0, q_1\}, \quad \delta(q_0, 1) = \{q_0\}, \quad \delta(q_1, 1) = \{q_2\},$$

其它转移为空集。直觉是： q_0 任意循环，看到某个 0 时非确定性猜它是倒数第二个字符。

Subset construction 的可达集合如下。令

$$A = \{q_0\}, \quad B = \{q_0, q_1\}, \quad C = \{q_0, q_2\}.$$

DFA state	on 0	on 1	accepting?
$A = \{q_0\}$	B	A	no
$B = \{q_0, q_1\}$	B	C	no
$C = \{q_0, q_2\}$	B	A	yes

于是 DFA 为

$$Q_D = \{A, B, C\}, \quad q_D = A, \quad F_D = \{C\},$$

转移表如上。注意 C 接受是因为 $C \cap \{q_2\} \neq \emptyset$ ，不是因为集合中所有 NFA 状态都接受。

2.5 NFA 转 DFA 示例 2: 带 ϵ -closure

设 NFA 识别语言 $\{a, ab\}$ 。状态

$$Q = \{s, x, y, z, w, u\}, \quad \Sigma = \{a, b\}, \quad q_0 = s, \quad F = \{y, u\}.$$

转移为:

$$\delta(s, \epsilon) = \{x, z\}, \quad \delta(x, a) = \{y\}, \quad \delta(z, a) = \{w\}, \quad \delta(w, b) = \{u\}.$$

其它转移为空。先算起始闭包:

$$E(\{s\}) = \{s, x, z\}.$$

令

$$A = \{s, x, z\}, \quad B = \{y, w\}, \quad C = \{u\}, \quad D = \emptyset.$$

DFA state	on a	on b	accepting?
$A = \{s, x, z\}$	B	D	no
$B = \{y, w\}$	D	C	yes
$C = \{u\}$	D	D	yes
$D = \emptyset$	D	D	no

解释其中一行:

$$\delta_D(A, a) = E(\delta(x, a) \cup \delta(z, a)) = E(\{y, w\}) = \{y, w\} = B.$$

B 和 C 都接受，因为它们分别包含 y 或 u 。这样得到的 DFA 精确接受 a 和 ab 。

2.6 正则表达式

正则表达式递归定义:

- $\emptyset, \epsilon, a$ 是正则表达式。
- 若 R_1, R_2 是正则表达式，则 $R_1 \cup R_2$ 、 $R_1 R_2$ 、 R_1^* 是正则表达式。

Kleene theorem:

$$\text{regular language} \iff \text{DFA/NFA} \iff \text{regular expression.}$$

2.7 Thompson 构造与状态消除

RE $\rightarrow$ NFA: 对基础表达式建小 NFA, 再递归处理 union/concat/star。

DFA/NFA $\rightarrow$ RE: 转 GNFA 后消状态。消去状态 q 时, 对任意 i, j 更新边标签:

$$R_{ij} \leftarrow R_{ij} \cup R_{iq}(R_{qq})^*R_{qj}.$$

作业中必须写添加新 start、新 accept、逐步消状态, 不能只给最终正则式。

2.8 Assignment 2 对齐

NFA 构造要点:

- ends with 00: 三态; 起点可在任意处猜测倒数两个 0 的开始。
- contains 0101: 五态; 匹配进度 0 到 4。
- even number of 0s or exactly two 1s: 用新起点 ϵ 分支到两个子 NFA。
- $\{0\}$: 两态, 读一个 0 接受。
- $0^*1^*0^+$: 三态, 注意最后的 0^+ 至少一个 0。
- $1^*(001^+)^*$: 起点接受, 读 00 后必须至少一个 1 回到起点。
- $\{\epsilon\}$: 单个起始接受态, 无其它转移。
- 0^* : 单个起始接受态, 0 自环。

Closure construction 题必须按证明构造:

- Union: 新起点 ϵ 到两个起点。
- Concatenation: 第一个接受态 ϵ 到第二个起点。
- Star: 新起点接受, 并从旧接受态 ϵ 回旧起点。
- Single accept state: 新增 q_a , 旧接受态 ϵ 到 q_a 。

Subset construction 题要列子集状态表。答案源码里示例:

- 无 ϵ 的 NFA: 状态集形如 $\emptyset, \{1\}, \{2\}, \{1, 2\}$ 。
- 有 ϵ 的 NFA: 起始态是 ϵ -closure, 例如 $\{1, 2\}$, 接受态为包含旧接受态的子集。

Bonus: 语言 “不含一对相隔奇数个符号的 1”。答案思路是先构造补语言 NFA: 猜第一个 1, 用两个状态记录中间间隔奇偶, 若在奇数间隔后看到第二个 1 则接受; 再取补得到原语言 DFA。

2.9 Assignment 3 对齐

正则表达式速记:

- starts with 1, ends with 0: $1\Sigma^*0$
- at least three 1s: $\Sigma^*1\Sigma^*1\Sigma^*1\Sigma^*$

- contains 0101: $\Sigma^*0101\Sigma^*$
- third symbol is 0: $\Sigma^20\Sigma^*$
- length at most 5: $(\Sigma \cup \epsilon)^5$
- strings of positive length: Σ^+
- empty language: $\emptyset$

作业中的 state elimination 示例:

- 对两态 automaton, 先加 s 和 f , 若 1 到 2 为 b , 2 自环 a , 2 回 1 为 b , 1 自环 a , 最终答案形如

$$(a \cup ba^*b)^*ba^*.$$

注释语言 C : 以 $/\#$ 开始, 以 $\#/$ 结束, 中间不能提前出现 $\#/$ 。答案给的正则式可简化记为:

$$/\#(\#^*(a \cup b) \cup /)^*\#^+ /$$

关键状态含义: 在 body 中看到 $\#$ 后, 若下一个是 $/$ 就结束; 若是 $a, b, \#$ 则继续。

3. 非正则语言与 Pumping Lemma

3.1 正则 Pumping Lemma

若 L 正则, 则存在 p , 任意 $w \in L, |w| \geq p$, 可写为 $w = xyz$, 满足:

$$|xy| \leq p, \quad |y| > 0, \quad \forall i \geq 0, xy^iz \in L.$$

它只能证明“非正则”, 不能证明“正则”。

3.2 Assignment 4 标准证明

**** $\{1^n2^n3^n \mid n \geq 0\}$ 非正则 ****

取 $w = 1^p2^p3^p$ 。由 $|xy| \leq p$, $y = 1^\ell, \ell > 0$ 。泵 $i = 2$ 后得到 $1^{p+\ell}2^p3^p$, 三段数量不相等, 矛盾。

**** $\{www \mid w \in \{a, b\}^*\}$ 或答案中 $(a^pb^p)^3$ 类型 ****

取 $w = (a^pb^p)^3$ 。由于 $|xy| \leq p$, y 位于第一段 a 中, 泵后第一段变长, 不能再分成三个完全相同的块, 矛盾。

**** $\{a^{2^n} \mid n \geq 0\}$ 非正则 ****

取 $w = a^{2^p}$, $y = a^\ell$, 且 $1 \leq \ell \leq p$ 。泵 $i = 2$ 后长度为 $2^p + \ell$, 满足

$$2^p < 2^p + \ell \leq 2^p + p < 2^{p+1}.$$

长度夹在连续两个 2 的幂之间, 矛盾。

**** 为什么不能用 0^p1^p 证明 0^*1^* 非正则 ****

要证明目标语言 0^*1^* 非正则, 泵后必须离开 0^*1^* 。但泵 0^p1^p 的前段 0 后仍是 0^*1^* 。你证明的是不在 $\{0^k1^k\}$, 不是不在 0^*1^* 。

**** $\{0^m1^m0^m \mid m \geq 0\}$ 非正则 ****

取 $w = 0^p1^p0^p$ 。 y 在第一段 0 内, 泵 $i = 2$ 后前后 0 数量不同。

**** $\{0^m 1^n \mid m \neq n\}$ 非正则 ****

用闭包：若 L 正则，则 $\bar{L} \cap 0^* 1^*$ 正则。但

$$\bar{L} \cap 0^* 1^* = \{0^n 1^n \mid n \geq 0\}$$

非正则，矛盾。

**** 出现 01 次数等于 10 次数的语言是正则 ****

关键 claim：次数相等 iff $w \in \{\epsilon, 0, 1\}$ 或首尾字符相同。

正则表达式：

$$\{\epsilon, 0, 1\} \cup 0\Sigma^*0 \cup 1\Sigma^*1.$$

**** 语言 $C = \{1^k y \mid y \text{ 中 } 1 \text{ 的个数} \leq k\}$ 非正则 ****

取 $w = 1^p 0 1^p$ ，前缀 1^p 作 $k = p$ 。泵掉前面的 $y = 1^\ell$ 后，开头最多 $p - \ell$ 个 1，但后缀仍有 p 个 1，违反“后缀 1 的个数 $\leq k$ ”。

4. CFG、CFL、PDA

4.1 CFG 定义

CFG 为

$$G = (V, \Sigma, R, S)$$

其中 V 是变量， Σ 是终结符， R 是产生式， S 是起始变量。

若一个字符串有两棵不同 parse tree，则文法 ambiguous。

4.2 常用 CFG

Assignment 5 答案中的典型文法：

- 至少三个 1：

$$S \rightarrow R1R1R1R, \quad R \rightarrow 0R \mid 1R \mid \epsilon.$$

- 以相同符号开始和结尾：

$$S \rightarrow 1R1 \mid 0R0 \mid 0 \mid 1, \quad R \rightarrow 0R \mid 1R \mid \epsilon.$$

- 奇数长度：

$$S \rightarrow 0S0 \mid 0S1 \mid 1S0 \mid 1S1 \mid 0 \mid 1.$$

- 奇数长度且中间符号为 0：

$$S \rightarrow 0S0 \mid 0S1 \mid 1S0 \mid 1S1 \mid 0.$$

- 回文：

$$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon.$$

- 空语言：可用 $S \rightarrow S$ 或无接受路径的 PDA。

4.3 CNF

Chomsky Normal Form 只允许:

$$A \rightarrow BC, \quad A \rightarrow a$$

如果语言含 ϵ , 允许新起始符号 $S_0 \rightarrow \epsilon$ 。

转换步骤:

1. 加新起始变量 $S_0 \rightarrow S$ 。
2. 消除 ϵ -productions。
3. 消除 unit productions。
4. 长右部拆成二元变量, 终结符替换成单独变量。

Assignment 5 Q6 示例:

原文法:

$$A \rightarrow BAB \mid B \mid \epsilon, \quad B \rightarrow 11 \mid \epsilon.$$

加新起始符号, 消 ϵ , 消 unit 后, 引入 $D \rightarrow 1, C \rightarrow AB$, 得到类似:

$$A_0 \rightarrow BC \mid BA \mid AB \mid BB \mid DD \mid \epsilon, \quad A \rightarrow BC \mid BA \mid AB \mid BB \mid DD, \quad C \rightarrow AB, \quad B \rightarrow DD, \quad D \rightarrow 1.$$

4.4 PDA

PDA 有输入、有限控制和栈。转移记为:

$$a, X \rightarrow \gamma$$

表示读入 a (可为 ϵ), 弹出 X (可为空, 按课件记法), 压入 γ 。

核心等价:

$$\text{CFL} \iff \text{某 PDA 识别.}$$

Assignment 5 中正则语言对应的 PDA 可不使用栈; 非正则 CFL 如 palindrome、奇数长度中间为 0 需要栈:

- palindrome: 非确定性猜中点, 前半段压栈, 后半段逐字符匹配弹栈。
- 奇数长度中间为 0: 先压前半段计数, 读中间 0, 后半段每读一个符号弹一个计数符号。

4.5 CFG 转 PDA: 完整构造

要证明每个 CFG 都有等价 PDA, 答案必须写出 PDA 如何模拟最左推导。给定

$$G = (V, \Sigma, R, S),$$

构造 PDA P , 使栈里保存“还需要匹配或展开的 sentential form”。复习时统一把栈顶写在最左边。

PDA 的状态可取

$$Q = \{q_{\text{start}}, q, q_{\text{acc}}\},$$

栈字母表

$$\Gamma = V \cup \Sigma \cup \{\$ \}.$$

核心转移如下：

1. 初始化：先把起始变量放到栈顶。

$$q_{\text{start}} \xrightarrow{\epsilon, \$ \rightarrow S\$} q.$$

2. 展开变量：对每条产生式

$$A \rightarrow X_1 X_2 \cdots X_k$$

加一条“用右部替换栈顶变量”的转移。若图或形式化定义只能一次压一个符号，则按

$$X_k, X_{k-1}, \dots, X_1$$

的顺序压栈，这样最后 X_1 在栈顶。简写为

$$q \xrightarrow{\epsilon, A \rightarrow X_1 X_2 \cdots X_k} q.$$

若 $A \rightarrow \epsilon$ ，则加

$$q \xrightarrow{\epsilon, A \rightarrow \epsilon} q.$$

3. 匹配终结符：对每个 $a \in \Sigma$ ，加

$$q \xrightarrow{a, a \rightarrow \epsilon} q.$$

意思是输入下一个符号和栈顶终结符相同，二者同时消掉。

4. 接受：输入读完且栈顶回到底符号时接受：

$$q \xrightarrow{\epsilon, \$ \rightarrow \epsilon} q_{\text{acc}}.$$

正确性句子可以这样写：PDA 每次用 $A \rightarrow u$ 替换栈顶变量，正好对应 CFG 最左推导中的一步；每次读入终结符 a 并弹出 a ，正好确认当前推导出的最左终结符与输入一致。因此 PDA 恰好接受所有可由 S 推导出的字符串。

4.6 CFG 转 PDA 示例： $0^n 1^n$

文法

$$S \rightarrow 0S1 \mid \epsilon$$

生成

$$\{0^n 1^n \mid n \geq 0\}.$$

按上面的构造，PDA 初始化时把 S 压到栈顶。展开规则为：

$$\epsilon, S \rightarrow 0S1, \quad \epsilon, S \rightarrow \epsilon.$$

若只能逐个压栈，实现 $S \rightarrow 0S1$ 时按 $1, S, 0$ 的顺序压，使得栈顶从左到右显示为 $0S1$ 。终结符匹配规则为：

$$0, 0 \rightarrow \epsilon, \quad 1, 1 \rightarrow \epsilon.$$

输入 0011 的一次接受计算可以写成“栈内容变化”：

$$S\$ \Rightarrow 0S1\$ \Rightarrow 00S11\$ \Rightarrow 0011\$ \Rightarrow 011\$ \Rightarrow 11\$ \Rightarrow 1\$ \Rightarrow \$.$$

前三步是展开 S ，后四步是读入并匹配输入。最后弹出 $\$$ 进入接受态。

4.7 PDA 转 CFG：完整构造

要证明每个 PDA 都有等价 CFG，答案通常先把 PDA 规范化，再引入变量 A_{pq} 。这是 Assignment 答案里最容易漏分的一类 construction proof。

先把 PDA 改成等价的规范形式：

- 只有一个接受态 q_{acc} 。
- 接受前栈必须为空。
- 每个转移只做一种栈操作：要么 push 一个栈符号，要么 pop 一个栈符号。若原转移替换多个符号，用中间状态拆开。

设规范化后的 PDA 状态集为 Q 。为每一对状态 $p, q \in Q$ 建一个 CFG 变量

$$A_{pq}.$$

含义必须写清楚：

A_{pq} generates exactly the strings that take the PDA from p with empty stack to q with empty stack.

起始变量是

$$A_{q_0 q_{acc}}.$$

产生式分三类：

1. 空计算：

$$A_{pp} \rightarrow \epsilon$$

对每个 $p \in Q$ 都加入。

2. 串接计算：

$$A_{pq} \rightarrow A_{pr} A_{rq}$$

对所有 $p, r, q \in Q$ 都加入。这表示从 p 到 q 的空栈计算可以在某个中间状态 r 分成两段。

3. push-pop 配对：若 PDA 有

$$p \xrightarrow{a, \epsilon \rightarrow t} r$$

和

$$s \xrightarrow{b, t \rightarrow \epsilon} q,$$

其中 $a, b \in \Sigma \cup \{\epsilon\}$, 则对所有这样的配对加入

$$A_{pq} \rightarrow aA_{rs}b.$$

解释：第一步 push 的 t 必须由最后一步 pop 掉，中间从 r 到 s 的计算不能动到这个 t ，所以中间部分由 A_{rs} 生成。

正确性句子可以固定写：任何从空栈到空栈的接受计算，要么是空计算，要么能在某个中间状态拆成两段，要么第一步 push 的栈符号与最后一次对应 pop 配对；三类产生式正好覆盖这三种情况。反过来，每条产生式都模拟 PDA 的合法计算片段，所以 CFG 不会生成额外字符串。

4.8 PDA 转 CFG 示例：从 PDA 得到 $S \rightarrow 0S1 \mid \epsilon$

构造一个规范 PDA 识别

$$\{0^n 1^n \mid n \geq 0\}.$$

状态为 p, r, s ，从 p 开始，接受状态为 s ，接受时栈为空。转移如下：

$$p \xrightarrow{0, \epsilon \rightarrow X} p, \quad p \xrightarrow{\epsilon, \epsilon \rightarrow Y} r, \quad r \xrightarrow{\epsilon, Y \rightarrow \epsilon} s, \quad s \xrightarrow{1, X \rightarrow \epsilon} s.$$

含义：在 p 中每读一个 0 就 push 一个 X ；用 push Y 、pop Y 的配对完成从 push 阶段到 pop 阶段的切换；在 s 中每读一个 1 就 pop 一个 X 。

按构造，起始变量是 A_{ps} 。由 Y 的 push-pop 配对：

$$p \xrightarrow{\epsilon, \epsilon \rightarrow Y} r, \quad r \xrightarrow{\epsilon, Y \rightarrow \epsilon} s$$

得到

$$A_{ps} \rightarrow \epsilon A_{rr} \epsilon.$$

又因为 $A_{rr} \rightarrow \epsilon$ ，所以有

$$A_{ps} \rightarrow \epsilon.$$

由 X 的 push-pop 配对：

$$p \xrightarrow{0, \epsilon \rightarrow X} p, \quad s \xrightarrow{1, X \rightarrow \epsilon} s$$

得到

$$A_{ps} \rightarrow 0A_{ps}1.$$

因此与起始变量有关的核心规则就是

$$A_{ps} \rightarrow 0A_{ps}1 \mid \epsilon,$$

这正是生成 $0^n 1^n$ 的 CFG。考试中可以把 A_{ps} 重命名为 S ，写成

$$S \rightarrow 0S1 \mid \epsilon.$$

4.9 CFL 闭包与非闭包

CFL 对并、连接、Kleene star 封闭。证明可用 CFG 或 PDA:

- 并: 新起始变量 $S \rightarrow S_1 \mid S_2$ 。
- 连接: $S \rightarrow S_1 S_2$ 。
- Star: $S \rightarrow S_1 S \mid \epsilon$ 。

每个正则语言都是 CFL: 对正则表达式结构归纳, 基础语言 $\emptyset, \epsilon, \{a\}$ 都有 CFG, CFL 又对正则运算封闭。

CFL 不对交和补封闭:

- 取

$$A = \{a^m b^n c^n \mid m, n \geq 0\}, \quad B = \{a^n b^n c^m \mid m, n \geq 0\}.$$

二者都是 CFL, 但

$$A \cap B = \{a^n b^n c^n \mid n \geq 0\}$$

不是 CFL。

- 若 CFL 对补封闭, 则由 De Morgan 与并封闭可推出对交封闭, 矛盾。

CFL 与正则语言的交封闭: 给 PDA P 和 DFA D , 构造乘积 PDA, 状态为 (q_P, q_D) , 栈行为照 PDA, 读入符号时同时更新 DFA; 接受态为 $F_P \times F_D$ 。

4.10 CFL Pumping Lemma

若 L 是 CFL, 则存在 p , 任意足够长 $s \in L$ 可写为

$$s = uvxyz$$

满足

$$|vxy| \leq p, \quad |vy| \geq 1, \quad \forall i \geq 0, \quad uv^i xy^i z \in L.$$

Assignment 6 标准证明:

- $\{0^n 1^n 0^n 1^n\}$ 非 CFL: 取 $0^p 1^p 0^p 1^p$ 。因 $|vxy| \leq p$, 只影响至多两个相邻块。泵后至少一个块长度变, 另有块保持 p , 四段无法同为 n 。
- $\{0^n \# 0^{2n} \# 0^{3n}\}$ 非 CFL: 取 $0^p \# 0^{2p} \# 0^{3p}$ 。 vxy 不可能跨两个 $\#$, 只改变局部一段或相邻两段, 泵后比例 $1:2:3$ 被破坏。

4.11 CNF 推导长度

若 CFG 在 CNF 中, 推导长度为 $|w|$ 的非空串时恰好需要

$$2|w| - 1$$

步: $|w| - 1$ 次 $A \rightarrow BC$ 让变量叶子数从 1 增到 $|w|$, 再 $|w|$ 次 $A \rightarrow a$ 生成终结符。

5. Turing Machines 与 Church-Turing

5.1 TM 定义

标准 TM:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

其中 Γ 是 tape alphabet, $\sqcup \in \Gamma$, $\Sigma \subseteq \Gamma - \{\sqcup\}$,

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}.$$

配置写作 uqv : 带内容为 uv , 当前状态 q , 读写头在 v 的第一个字符上。

5.2 Decider vs Recognizer

- Decider: 所有输入都 halt, 接受 L 中输入, 拒绝非 L 输入。
- Recognizer: L 中输入会 accept; 非 L 输入可以 reject 或 loop。
- Decidable $\Rightarrow$ Turing-recognizable。
- A decidable iff A 和 $\bar{A}$ 都 Turing-recognizable。

5.3 TM 变体

PPT 覆盖的等价模型:

- 多带 TM 等价于单带 TM。
- NTM 等价于 DTM (可模拟所有计算分支)。
- Enumerator 与 TM 识别器等价。
- Church-Turing thesis: 直观可算法计算的函数可由 TM 计算。

作业中重要变体:

****2-PDA 识别 $\{a^n b^n c^n\}$ ****

两个栈即可模拟三段相等: 读 a 时压 stack1; 读 b 时压 stack2; 读 c 时同时弹两个栈; 输入结束且两栈空则接受。这说明 2-PDA 比 1-PDA 强。

Left-reset TM 模拟普通左移

普通 TM 想从当前格左移时, left-reset machine 用标记法:

1. 标记当前格, reset 到最左端。
2. 从左端开始逐步找当前格前一格。
3. 用额外标记推进“候选前驱”。
4. 当候选右边是原当前格, 擦标记并停在前驱格。

答案强调: 要写出这个算法过程, 而不是只说“可以模拟”。

只能右移/停留或输入只读的 TM

这类机器不能反复利用输入位置的无限记忆, 识别能力退化为正则语言。证明思路: 构造等价有限自动机, 状态编码 TM 从当前位置向右移动后的有限控制状态。

5.4 TM 设计算法题

Assignment 7 典型题:

- 识别 $\{a^n b^n c^n\}$: 循环扫描, 找未标记 **a** 改为 **X**, 向右找未标记 **b** 改为 **Y**, 再找未标记 **c** 改为 **Z**, 回到左端; 最后确认没有未标记 **a, b, c** 且顺序合法。
- $\#0 = \#1$: 反复找一个未标记 0 和一个未标记 1 配对标记; 若最后只剩标记则接受。
- $\#0 = 2\#1$: 每找一个 1, 匹配两个 0。
- 补语言: 若已有 decider, 可翻转 accept/reject; 若只是 recognizer, 不能直接补。
- 二进制减一: 移到最右, 从右向左把连续 0 改成 1, 遇到第一个 1 改成 0 后停。
- 二进制减法 $x - y$: 重复对 x 和 y 执行减一, 直到 $y = 0$ 。
- 整数除法 $\lfloor x/y \rfloor$: 多带 TM 上重复减 y , 计数 quotient。

6. Decidability 与 Undecidability

6.1 可判定语言

经典可判定问题:

- $A_{DFA} = \{\langle M, w \rangle \mid M \text{ is a DFA and accepts } w\}$
- A_{NFA}
- A_{REX}
- $E_{DFA} = \{\langle M \rangle \mid L(M) = \emptyset\}$
- EQ_{DFA}
- A_{CFG}
- E_{CFG}

典型算法:

- A_{DFA} : 直接模拟 DFA。
- A_{NFA} : 转 DFA 或图搜索所有 NFA 分支。
- A_{REX} : $RE \rightarrow NFA \rightarrow DFA$, 再判定。
- E_{DFA} : 从 start 做 BFS, 看是否能到接受态。
- EQ_{DFA} : 构造对称差 DFA, 判空。
- A_{CFG} : 转 CNF 后 CYK/dynamic programming。
- E_{CFG} : 标记能生成终结串的变量, 看 start 是否被标记。

6.2 Assignment 8 对齐

DFA 与 REX 等价可判定

对输入 $\langle M, R \rangle$:

1. 将 R 转为 NFA。

2. 将 NFA 转为 DFA M' 。
3. 运行 EQ_{DFA} 于 $\langle M, M' \rangle$ 。

**** ALL_{DFA} 可判定 ****

构造识别 $\overline{L(A)}$ 的 DFA M ，运行 E_{DFA} 。若补语言为空，则 $L(A) = \Sigma^*$ 。

**** A_{CFG} 可判定 ****

运行 A_{CFG} 于 $\langle G, \epsilon \rangle$ 。或转 CNF 后检查是否有 $S \rightarrow \epsilon$ 。

**** $\overline{E_{TM}}$ Turing-recognizable ****

对输入 $\langle M \rangle$ ，枚举所有字符串 $s_1, s_2, \dots$ ，dovetail 模拟：第 i 轮让前 i 个字符串各跑 i 步。一旦某个输入被接受，就接受。若 $L(M) = \emptyset$ ，机器可能永远不接受。

**** EQ_{DFA} 有限测试上界 ****

若 A, B 有 n_1, n_2 个状态，则对称差 DFA 有 $n_1 n_2$ 个状态。若存在区分串，则存在长度小于 $n_1 n_2$ 的区分串。需测试数量：

$$\sum_{i=0}^{n_1 n_2 - 1} |\Sigma|^i.$$

6.3 Diagonalization 与 A_{TM}

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$$

是 Turing-recognizable 但 undecidable。

Recognizer: 通用 TM 模拟 $M(w)$ ，若 M accept 则 accept，若 reject 则 reject，若 loop 则 loop。

不可判定证明：

1. 假设 H decides A_{TM} 。
2. 构造 D ：输入 $\langle M \rangle$ ，运行 $H(\langle M, \langle M \rangle \rangle)$ 。若 H accept，则 D reject；若 H reject，则 D accept。
3. 运行 $D(\langle D \rangle)$ 得矛盾。

$\overline{A_{TM}}$ 不是 Turing-recognizable。否则 A_{TM} 和其补都 recognizable，可并行运行得到 decider，矛盾。

6.4 Reducibility

映射归约：

$$A \leq_m B$$

若存在可计算函数 f ，使得

$$w \in A \iff f(w) \in B.$$

性质：

- 若 $A \leq_m B$ 且 B decidable，则 A decidable。
- 若 $A \leq_m B$ 且 A undecidable，则 B undecidable。
- 若 $A \leq_m B$ 且 B T-recognizable，则 A T-recognizable。
- 若 $A \leq_m B$ 且 A T-unrecognizable，则 B T-unrecognizable。

6.5 Rice Theorem

任何关于 $L(M)$ 的非平凡性质都是 undecidable。

非平凡：有些 TM 的语言满足该性质，有些不满足。

注意：Rice 只适用于“语言性质”，不适用于“TM 语法性质”如状态数是否大于 481。

Assignment 10 中用 Rice 证明：

- $L(M)$ infinite：非平凡语言性质，undecidable。
- $1011 \in L(M)$ ：非平凡语言性质，undecidable。
- $L(M) = \Sigma^*$ ：非平凡语言性质，undecidable。

6.6 Assignment 9-10 对齐

不可数性

$\{0, 1, 2\}^{\mathbb{N}}$ 不可数：假设列成 $f(0), f(1), \dots$ ，构造新序列 s ，第 i 位取不同于 $f(i)$ 第 i 位的符号。则 s 与每一行在对角位不同。

三元组集合可数

按 $i + j + k = n$ 分层，每层有限：

$$\binom{n+2}{2}$$

先列和小的，再列同层内部，得到枚举。

“TM 是否会在空白带上写非空符号”可判定

如果在写非空符号前，带一直全 blank，行为只由状态决定。模拟 $|Q| + 1$ 步：若没写且状态重复，则之后永远重复，不会写。

“TM 是否至少 481 个状态”可判定

这是语法性质，直接解析编码并计数状态。

DFA 是否不接受任何偶数个 1 的串

构造 DFA A 接受偶数个 1 的串，构造 $M \cap A$ ，运行 E_{DFA} 判空。

**** $\overline{EQ_{CFG}}$ Turing-recognizable****

枚举所有字符串 w ，运行 A_{CFG} 判断 w 是否由 G_1, G_2 生成；若结果不同则接受。若两个 CFG 等价，则永不接受。因此 EQ_{CFG} 是 co-Turing-recognizable。

****若 A T-recognizable 且 $A \leq_m \bar{A}$ ，则 A decidable****

由归约 f 得：

$$w \in \bar{A} \iff f(w) \in A.$$

因为 A recognizable，先计算 $f(w)$ ，再运行 A 的 recognizer，即可 recognize $\bar{A}$ 。所以 A 与 $\bar{A}$ 都 recognizable，故 A decidable。

判定 halts on empty input 不可判定

从 $HALT_{TM}$ 归约。给 $\langle M, w \rangle$ ，构造 M' ：在空输入上模拟 $M(w)$ ，若 $M(w)$ halt 则 halt，否则 loop。若能判定 M' 是否在空输入 halt，就能判定 $HALT_{TM}$ 。

7. Time Complexity 与 P

7.1 时间复杂度

TM 在时间 $t(n)$ 内运行：对所有长度 n 输入，最多 $t(n)$ 步内 halt。

$$TIME(t(n)) = \{B \mid B \text{ 可由确定性单带 TM 在 } O(t(n)) \text{ 时间判定}\}.$$

$$P = \bigcup_k TIME(n^k)$$

即多项式时间可判定语言。

课件强调：

- 最坏情况复杂度是对长度 n 的所有输入取上界。
- reasonable encoding 很重要：图的 adjacency matrix/list 是合理编码，数字用 unary 通常不合理。
- 多带 TM 与单带 TM 在多项式意义下等价：多带 $t(n)$ 可由单带 $O(t(n)^2)$ 模拟。

7.2 P 中的典型问题

****PATH $\in P$ ****

给有向图 G, s, t ，从 s 做 BFS/markings，若标记到 t 则接受。

****RELPRIME $\in P$ ****

用 Euclidean algorithm：重复 $x \leftarrow x \bmod y$ ，交换 x, y ，直到 $y = 0$ 。若 gcd 为 1 则接受。

****CFL $\in P$ ****

用 CYK/dynamic programming 而不是暴力枚举推导。转 CNF 后，对每个 substring 存能生成它的变量集合，总复杂度多项式，典型为 $O(n^3)$ 级别乘文法大小因子。

HAMPATH 与 PATH 的区别

PATH 只问是否存在任意路径，BFS 可解；HAMPATH 要经过每个节点恰好一次，暴力路径数可达 $m!$ ，是否在 P 是开放/与 NP 完全性相关。

7.3 P 闭包与 Assignment 10

P 对并、连接、补封闭。

- Union：顺序运行两个多项式 decider，任一接受则接受。
- Concatenation：枚举 $n+1$ 个 split $w = uv$ ，运行两个 decider，某个 split 都接受则接受。
- Complement：运行 decider 并翻转结果。

****CONNECTED $\in P$ ****

无向图从任意顶点 BFS，若所有顶点都被标记则接受。

****ALL_DFA $\in P$ ****

构造补 DFA，对补 DFA 从起点 BFS。若能到接受态，则原 DFA 不是 all；否则原 DFA 接受 Σ^* 。

****EQ_DFA $\in P$ ****

构造对称差 DFA，再用 E_{DFA} 的 BFS 判空。

8. NP、coNP、Polynomial Reducibility

8.1 NP 定义

$$NP = \bigcup_k NTIME(n^k)$$

等价定义：存在多项式时间 verifier V 和多项式长度 certificate c ，使得

$$w \in L \iff \exists c, |c| \leq |w|^k, V(w, c) \text{ accepts.}$$

PPT 中重要定理：语言在 NP 中 iff 有 polynomial-time verifier。

NTM $\rightarrow$ verifier: certificate 描述接受分支。

verifier $\rightarrow$ NTM: 非确定性猜 certificate，再运行 verifier。

8.2 常见 NP 证明

Assignment 11 要求证明这些语言在 NP。标准写法是给 certificate 和 verifier:

- 3SAT: certificate 是 truth assignment; 检查每个 clause 是否有 true literal。
- VERTEX-COVER: certificate 是 k 个顶点; 检查每条边至少一个端点在集合中。
- TSP: certificate 是 tour; 检查每个城市恰好一次且总权重 $\leq k$ 。
- MAX-CUT: certificate 是顶点划分 $S, V - S$; 数 crossing edges 是否 $\geq k$ 。
- 3-COLORING: certificate 是每个顶点颜色; 检查相邻顶点颜色不同。
- CLIQUE: certificate 是 k 个顶点; 检查任意两点之间有边。
- SUBSET-SUM: certificate 是子集; 检查和是否为目标 t 。
- COMPOSITES: certificate 是非平凡因子 y ; 检查 $1 < y < x$ 且 $y \mid x$ 。

8.3 NP 闭包

NP 对 union、concatenation、star 封闭。

- Union: certificate 额外包含选择 bit，说明用 V_1 还是 V_2 。
- Concatenation: certificate 包含 split 位置 i 以及两个子证书 c_1, c_2 。
- Star: certificate 包含分割位置列表和每段证书; 段数至多 $|w|$ ，总长度仍多项式。

8.4 P、NP、coNP 判断题

标准结论:

- $P \subseteq NP$: True, 忽略 certificate, 直接运行 P decider。
- P 对补封闭: True。
- $P \subseteq NP \cap coNP$: True。
- 若 $P = NP$, 则 $NP = coNP$: True。
- $\overline{SAT} \in P$: 严格说是开放问题; 若它在 P, 则 $P = NP$ 。作业官方 revised answer 更严谨地说明 unknown; 旧简答里写 False 是基于通常相信 $P \neq NP$, 考试应按题目/答案口径说明 “unknown unless assuming $P \neq NP$ ”。

8.5 2-COLOR 与 2-SAT

****2-COLOR $\in P \cap NP$ ****

NP: certificate 是 2-coloring。

P: BFS/DFS 检查二分图。未染色点染 0, 邻居染 1, 若遇到同色边则 reject。

****2-SAT $\in P \cap NP$ ****

NP: certificate 是 assignment。

P: 构造 implication graph:

$$(x \vee y) \equiv (\neg x \rightarrow y) \wedge (\neg y \rightarrow x).$$

若某变量 x 与 $\neg x$ 在同一个 SCC, 则不可满足; 否则可满足。

9. NP-Completeness

9.1 定义与证明套路

语言 B 是 NP-complete iff:

1. $B \in NP$
2. 对所有 $A \in NP$, $A \leq_p B$

实际证明 C NP-complete:

1. 证明 $C \in NP$ 。
2. 从一个已知 NP-complete 问题 B 归约到 C , 如 $3SAT \leq_p C$ 。
3. 证明构造多项式时间。
4. 证明 iff。

9.2 3SAT $\rightarrow$ CLIQUE

给 3CNF:

$$\phi = C_1 \wedge \cdots \wedge C_m.$$

构造图 G :

1. 对每个 clause 中每个 literal occurrence 建一个顶点。
2. 不同 clause 的两个顶点之间连边, 当且仅当两个 literal 不矛盾。
3. 设置 $k = m$ 。Assignment 12 Q1 中 $m = 3$, 所以 $k = 3$ 。

正确性:

- 若 ϕ satisfiable, 从每个 clause 选一个 true literal。这些 literal 互不矛盾, 所以对应顶点两两相连, 形成 m -clique。
- 若 G 有 m -clique, 因为同 clause 内没有边, clique 必定每个 clause 选一个顶点; 两两相连说明 literal 互不矛盾, 可扩展为满足赋值。

9.3 2SAT -> CLIQUE

同样构造，每个 2-clause 建两个顶点，连接不同 clause 且矛盾的 literal，令 $k = m$ 。

能说明 2SAT 实例可多项式变成 CLIQUE 实例；但由于 2SAT 在 P，这个归约不能证明 CLIQUE NP-hard。要证明 CLIQUE NP-hard 必须从 NP-complete 问题如 3SAT 归约。

9.4 Double-SAT NP-complete

Double-SAT:

$$\{\phi \mid \phi \text{ has at least two satisfying assignments}\}.$$

在 NP: certificate 是两个不同 assignment，验证二者不同且都满足 ϕ 。

从 3SAT 归约：给 ϕ ，引入新变量 w ，构造

$$\phi' = \phi \wedge (w \vee \bar{w}).$$

若 ϕ satisfiable，则任意满足赋值可扩展为 $w = \text{true}$ 和 $w = \text{false}$ 两个满足赋值。若 ϕ' 有至少两个满足赋值，则特别地 ϕ satisfiable，从而 ϕ satisfiable。

9.5 3SAT -> HAMPATH

课件与作业采用 Sipser diamond variable gadget:

- 每个变量一个 gadget。
- 从左到右遍历表示 true，从右到左遍历表示 false。
- 每个 clause 一个 clause vertex，通过 wire 连到对应 literal 在 gadget 中的位置。
- 若某 literal 使 clause satisfied，Hamiltonian path 可绕入该 clause vertex 并返回。

证明结构：

- 满足赋值 -> 按变量真值选择每个 gadget 的遍历方向，对每个 clause 通过一个 true literal 的 wire 插入 clause vertex，得到 Hamiltonian path。
- Hamiltonian path -> 每个变量 gadget 的方向给出赋值；每个 clause vertex 被访问必须通过某个 literal wire，该 literal 在对应方向下为 true，所以每个 clause satisfied。

9.6 Directed HAMPATH -> Undirected HAMPATH

Assignment 13 Q4:

给 directed $G = (V, E)$ ，构造 undirected G' 。对每个顶点 u 建：

$$u^{in}, u^{mid}, u^{out}$$

并加内部边：

$$\{u^{in}, u^{mid}\}, \quad \{u^{mid}, u^{out}\}.$$

对每条有向边 (u, v) ，加无向边：

$$\{u^{out}, v^{in}\}.$$

起点终点:

$$s' = s^{in}, \quad t' = t^{out}.$$

关键证明: u^{mid} 度只连 u^{in}, u^{out} , 所以 Hamiltonian path 必须连续穿过一个 gadget。路径从 s^{in} 开始会强制所有 gadget 都以 $in \rightarrow mid \rightarrow out$ 方向穿过; 跨 gadget 的边只能对应原图有向边。

9.7 3SAT $\rightarrow$ VERTEX-COVER

Assignment 13 Q5:

给 3CNF ϕ , 变量数 v , clause 数 c 。

构造:

1. 每个变量 x_i : 建两个顶点 $x_i, \bar{x}_i$, 加边 $\{x_i, \bar{x}_i\}$ 。
2. 每个 clause $C_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$: 建三角形三个 clause 顶点。
3. 每个 clause literal 顶点连到变量 gadget 中同名 literal 顶点。
4. 设置

$$k = v + 2c.$$

正确性:

- 满足赋值 $\rightarrow$ 每个变量 gadget 选 true literal 顶点; 每个 clause 三角形中留下一个 true literal 顶点不选, 选另两个。变量边、三角形边、连接边都被覆盖, 大小 $v + 2c$ 。
- Vertex cover 大小 $\leq v + 2c \rightarrow$ 每个变量边至少选一个, 共至少 v ; 每个三角形至少选两个, 共至少 $2c$ 。因此必须恰好选这些数量。每个 clause 三角形有一个未选顶点, 其连接边必须由变量 gadget 中同名顶点覆盖, 令该 literal 为 true, 即每个 clause 有 true literal。

9.8 3COLOR NP-complete

Assignment 12 Q6:

在 NP: certificate 是 coloring, 逐边检查端点颜色不同。

从 3SAT 归约:

1. 建 palette 三角形 T, F, N , 强制三种颜色。
2. 每个变量建 $x_i, \bar{x}_i$, 二者互连, 且都连到 N 。因此它们不能取 Neutral, 且必须一真一假。
3. 每个 clause $(a \vee b \vee c)$ 用两个 OR gadget: 先算 $u = a \vee b$, 再算 $o = u \vee c$ 。
4. gadget 的中间输出和最终输出连到 N , 使其只能 True/False。
5. 最终输出 o 还连到 F , 强制 o 为 True。

正确性: clause 三个 literal 全 false 时 OR output 被迫 false, 与连到 F 冲突; 至少一个 true 时 gadget 可合法 3-color。

10. Cook-Levin 与 3SAT

10.1 Cook-Levin Theorem

SAT is NP-complete.

证明结构:

1. $SAT \in NP$: certificate 是 truth assignment。
2. 对任意 $A \in NP$, 取多项式时间 NTM M 决定 A 。给输入 w , 构造公式 $\phi_{M,w}$, 使

$$w \in A \iff \phi_{M,w} \text{ satisfiable.}$$

10.2 Tableau

若 M 在 n^k 时间内运行, 构造 $n^k \times n^k$ tableau, 每行是一条 configuration。变量:

$$x_{i,j,s}$$

表示 tableau 的第 i 行第 j 格内容是符号/状态 s 。

公式:

$$\phi_{M,w} = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}.$$

- ϕ_{cell} : 每个 cell 恰好一个符号。
- ϕ_{start} : 第一行是 $q_0 w \sqcup \dots$ 。
- ϕ_{accept} : 某处出现 q_{accept} 。
- ϕ_{move} : 每个 2×3 window 合法, 保证相邻配置符合转移函数。

10.3 为什么用 2×3 window

Assignment 13 Q3 反例:

设

$$\delta(q, 0) = (r, 0, L), \quad \delta(q, 1) = (s, 1, L), \quad r \neq s.$$

考虑 window:

$$\begin{array}{ccc} 0 & q & 1 \\ r & 0 & 1 \end{array}$$

它非法, 因为上方状态 q 实际扫描的是右侧 1, 应产生 s 而不是 r 。但它的两个 2×2 子窗口都可分别出现在某个合法 2×3 window 中。因此只查 2×2 不够。

10.4 Assignment 13 Q1 tableau

TM 识别 odd strings, 对 $w = 111$ 的计算:

$$q_0 111 \perp \vdash 1 q_1 11 \perp \vdash 11 q_0 1 \perp \vdash 111 q_1 \perp \vdash 111 \perp q_{acc}.$$

对应 5×5 tableau:

1	2	3	4	5
q_0	1	1	1	$\perp$
1	q_1	1	1	$\perp$
1	1	q_0	1	$\perp$
1	1	1	q_1	$\perp$
1	1	1	$\perp$	q_{acc}

$$\phi_{start} = x_{1,1,q_0} \wedge x_{1,2,1} \wedge x_{1,3,1} \wedge x_{1,4,1} \wedge x_{1,5,\perp}.$$

$$\phi_{accept} = \bigvee_{i=1}^5 \bigvee_{j=1}^5 x_{i,j,q_{acc}}.$$

10.5 SAT $\rightarrow$ 3SAT

3SAT 是 NP-complete。长 clause 转 3CNF:

- 1 literal: x 可写成 $(x \vee x \vee x)$ 。
- 2 literals: $(x \vee y)$ 可写成 $(x \vee y \vee x)$ 或用 padding。
- 3 literals: 保持。
- $l > 3$ literals:

$$(x_1 \vee \cdots \vee x_l)$$

替换为

$$(x_1 \vee x_2 \vee z_1) \wedge (\bar{z}_1 \vee x_3 \vee z_2) \wedge \cdots \wedge (\bar{z}_{l-3} \vee x_{l-1} \vee x_l).$$

注意课件说原公式与新公式不必逻辑等价，只需 satisfiability 等价。

11. Space, PSPACE, NP-hardness, 近似

11.1 Space complexity

TM 在 space $f(n)$ 内运行：对长度 n 输入，最多使用 $f(n)$ 个 tape cells。

$$SPACE(f(n)) = \{B \mid B \text{ 被某 deterministic TM 用 } O(f(n)) \text{ space 判定}\}$$

$$NSPACE(f(n)) = \{B \mid B \text{ 被某 nondeterministic TM 用 } O(f(n)) \text{ space 判定}\}.$$

$$PSPACE = \bigcup_k SPACE(n^k), \quad NPSPACE = \bigcup_k NSPACE(n^k).$$

Savitch theorem:

$$NSPACE(f(n)) \subseteq SPACE(f(n)^2), \quad f(n) \geq \log n.$$

推论:

$$PSPACE = NPSPACE.$$

课件关系:

$$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$$

且 $P \neq EXPTIME$, 所以这些包含中至少一个是真包含, 但不知道是哪一个。

11.2 NP-hardness

NP-hard 不要求问题在 NP 中, 也可用于 search/optimization 问题。

定义: 对所有 $A \in NP$, $A \leq_p B$ 。若某 NP-hard 问题有多项式算法, 则 $P = NP$ 。

NP-complete = NP-hard + in NP。

11.3 近似与随机算法

Vertex Cover 2-approximation

算法: 当图中还有边时, 任选一条边 (u, v) , 把 u, v 都加入 cover, 并删除所有 incident edges。

证明:

1. 输出 X 是 vertex cover, 因为每条边在被处理或删除时都被选中端点覆盖。
2. 选中的边彼此不共享端点, 任意最优 cover 至少要为每条这样的边选一个端点。
3. 算法每条选中边选两个端点, 所以 $|X| \leq 2OPT$ 。

****Randomized MAX-3SAT 7/8 expectation****

每个变量独立以 $1/2$ 取 true。一个 3-literal clause 不满足概率为

$$(1/2)^3 = 1/8$$

满足概率 $7/8$ 。令 X_i 是第 i 个 clause 是否满足的 indicator, 则

$$\mathbb{E}[X_i] = 7/8, \quad \mathbb{E}[X] = \sum_i \mathbb{E}[X_i] = 7m/8.$$

12. 快速总表

12.1 模型能力

模型	等价描述	语言类
DFA	NFA, regex	Regular
PDA	CFG	CFL
TM decider	总停机算法	Decidable
TM recognizer	接受时停机	Turing-recognizable
Poly-time DTM	efficient decider	P

Poly-time NTM / verifier	short certificate	NP
--------------------------	-------------------	----

12.2 常见“可判定/不可判定/可识别”

问题	结论
$A_{DFA}, A_{NFA}, A_{REG}$	decidable
E_{DFA}, EQ_{DFA}	decidable
A_{CFG}, E_{CFG}	decidable
A_{TM}	recognizable, undecidable
$\overline{A_{TM}}$	not recognizable
$HALT_{TM}$	undecidable
E_{TM}	undecidable, not recognizable
$\overline{E_{TM}}$	recognizable
EQ_{TM}	undecidable, not recognizable
$\overline{EQ_{CFG}}$	recognizable

12.3 常见 NP-complete 链

$$SAT \leq_p 3SAT \leq_p CLIQUE$$

$$3SAT \leq_p HAMPATH$$

$$HAMPATH \leq_p UHAMPATH$$

$$3SAT \leq_p VERTEX-COVER$$

$$3SAT \leq_p 3COLOR$$

12.4 作业对应复习索引

作业	必会内容
Assignment 1	DFA 形式化、补集、乘积构造
Assignment 2	NFA 图、union/concat/star construction、single accept、subset construction
Assignment 3	regex、Thompson、GNFA state elimination、reverse 正则性
Assignment 4	正则 pumping lemma、闭包反证
Assignment 5	CFG、parse tree、PDA、CNF
Assignment 6	CFG-PDA 等价、CFL closure、CFL pumping、CFL 非闭包、CFL $\cap$ regular
Assignment 7	2-PDA、TM 配置、TM 设计、left-reset、read-only/right-only 正则性、decidable/TR closure
Assignment 8	DFA/RE/CFG 判定问题、 $\overline{E_{TM}}$ recognizer、 EQ_{DFA} 长度上界

Assignment 9	对角线、可数性、TM 语法/行为判定、DFA 与偶数个 1
Assignment 10	EQ_{CFG} co-recognizable、mapping reduction、Rice、CYK、P 闭包、CONNECTED/ALL_DFA/EQ_DFA in P
Assignment 11	NP verifier、NP closure、2COLOR、2SAT、P/NP/coNP 判断
Assignment 12	3SAT- $\rightarrow$ CLIQUE、2SAT- $\rightarrow$ CLIQUE、Double-SAT、3SAT- $\rightarrow$ HAMPATH、3COLOR
Assignment 13	Cook-Levin tableau、legal window、UHAMPATH、VERTEX-COVER

13. 最后复习提醒

1. 自动机题：先写“状态记录什么”，再写状态/转移/接受态。
2. 构造证明题：必须给完整 tuple 或完整规则，不要只说“按闭包性”。
3. Pumping 题：选择的串必须在语言中，且分解必须任意。
4. 归约题：方向绝对不能反。从已知难问题归约到目标问题。
5. NP-complete 题：先 in NP，再 NP-hard。
6. Rice 题：先确认是 $L(M)$ 的非平凡语义性质，不是机器编码的语法性质。
7. P/NP 题：注意“可验证”与“可求解”不同；HAMPATH 在 NP，但不知道是否在 P。